



Audit report

Curvy 2026 Q1: Curvy Protocol - Smart Contracts

Authors:

Miroslav Stefanović, Mirel Dalčeković

05/08/2026

Contents

Contents	2
Curvy 2026 Q1 Project Overview.....	4
Scope.....	4
Relevant Code Commits.....	4
Conclusion	4
Curvy 2026 Q1 Audit Dashboard.....	5
Target Summary	5
Engagement Summary	5
Findings Severity Summary.....	5
Findings Category Breakdown	5
Curvy 2026 Q1 System overview	6
Overview	6
Curvy 2026 Q1 Invariant Analysis	9
Smart Contract Deployment and Upgrade	9
PortalFactory and Portal smart contracts.....	9
CurvyAggregatorAlphaV6.sol.....	12
CurvyVaultV6.sol.....	12
Bridging/Shielding flow	15
Curvy 2026 Q1 Findings	17
Curvy 2026 Q1 No upper limit for the fee.....	18
Curvy 2026 Q1 Deregistering token does not check the vault balance.....	19
Curvy 2026 Q1 EOA takeover	20
Curvy 2026 Q1 Burning balance during recovery.....	21
Curvy 2026 Q1 Reset function	22
Curvy 2026 Q1 Missing Smart Contract address check	23
Curvy 2026 Q1 Lost gas for transaction payment.....	24
Curvy 2026 Q1 Short-circuit guard bug/operator precedence bug	25
Curvy 2026 Q1 No Error Handling for Invalid JSON	26
Curvy 2026 Q1 Missing Null/Undefined Return Handling	28
Curvy 2026 Q1 No Validation of Address Format.....	29
Curvy 2026 Q1 Duplicated functionality for collecting fees.....	30
Curvy 2026 Q1 Wrong data in the event.....	31
Curvy 2026 Q1 EOA as tokenAddress	32
Curvy 2026 Q1 Collecting zero fees	33
Curvy 2026 Q1 Gas optimization	34
Curvy 2026 Q1 Moving constant to JSON.....	36

Curvy 2026 Q1 Deprecated fields	37
Curvy 2026 Q1 LiFi error message not propagated.....	39
Curvy 2026 Q1 No way to query which portals were deployed and when	40
Curvy 2026 Q1 Inconsistent Parameter Naming	41
Curvy 2026 Q1 Modifier instead of error.....	42
Curvy 2026 Q1 Findings Classification.....	44
Finding Categories.....	44
Severity Levels	44
Finding Severity Categories	44
Curvy 2026 Q1 Disclaimer	45

Curvy 2026 Q1 Project Overview

Scope

In March 2026, Curvy engaged [Ethereal](#) about a security audit of the Curvy Protocol smart contracts, more specifically:

- `0xCurvy/contracts/blob/main/contracts/aggregator-alpha/CurvyAggregatorAlphaV6.sol`
- `0xCurvy/contracts/blob/main/contracts/vault/CurvyVaultV6.sol`
- `0xCurvy/contracts/blob/main/contracts/portal/PortalFactory.sol`
- `0xCurvy/contracts/blob/main/contracts/portal/Portal.sol`
- `0xCurvy/contracts/blob/main/contracts/portal/SolanaPortal.sol`

The audit focused on evaluating the correctness and security properties of the implementation of the given Curvy Protocol components, including the deployment and upgrade process.

The audit was performed from March 23th, 2026 to April 15th, 2026 by the following personnel:

- Miroslav Stefanovic
- Mirel Dalcekovic

Relevant Code Commits

The audited code was from: [5f6e43c](#)

Conclusion

The Ethereal team finds the Curvy protocol's codebase to be generally well-structured, with clearly separated responsibilities between smart contracts. Processes of deployment and upgrade follow the well-known best practices.

In summary, this audit identified a total of 23 issues, without any Critical or High-severity vulnerabilities. The Curvy team successfully addressed all reported issues. Following a thorough review, the audit team confirmed that the implemented fixes are satisfactory.

Curvy 2026 Q1 Audit Dashboard

Target Summary

- Name: Curvy 2026 Q1: Curvy Protocol
- Code and specification version: 5f6e43cf153c0402bab7d1c041978ac8cf7a384b
- Platform: Solidity

Engagement Summary

- Dates: March 23th, 2026 to April 15th, 2026
- Method: Manual code review, protocol analysis

Findings Severity Summary

Finding Severity	#
Critical	0
High	0
Medium	6
Low	10
Informational	6
Total	22

Findings Category Breakdown

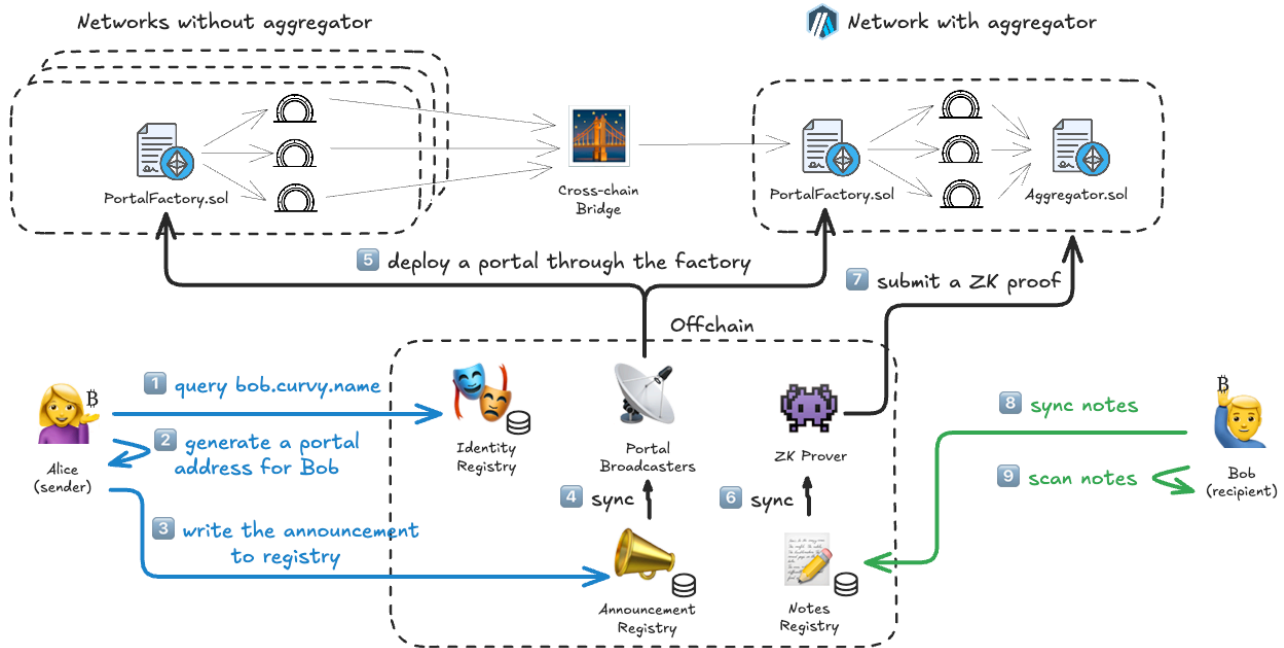
Finding Category	#
Protocol	0
Implementation	15
Optimization issue	1
Code improvement	6
Total	22

Curvy 2026 Q1 System overview

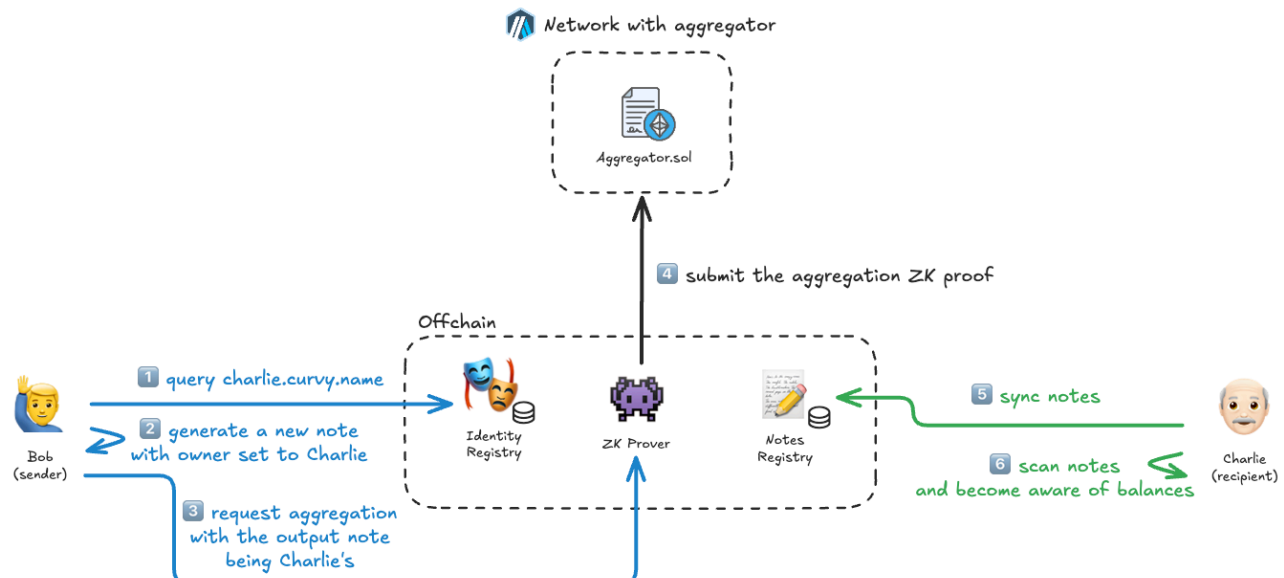
Overview

Curvy is a privacy solution for secure, fast, and compliant cryptocurrency payments.

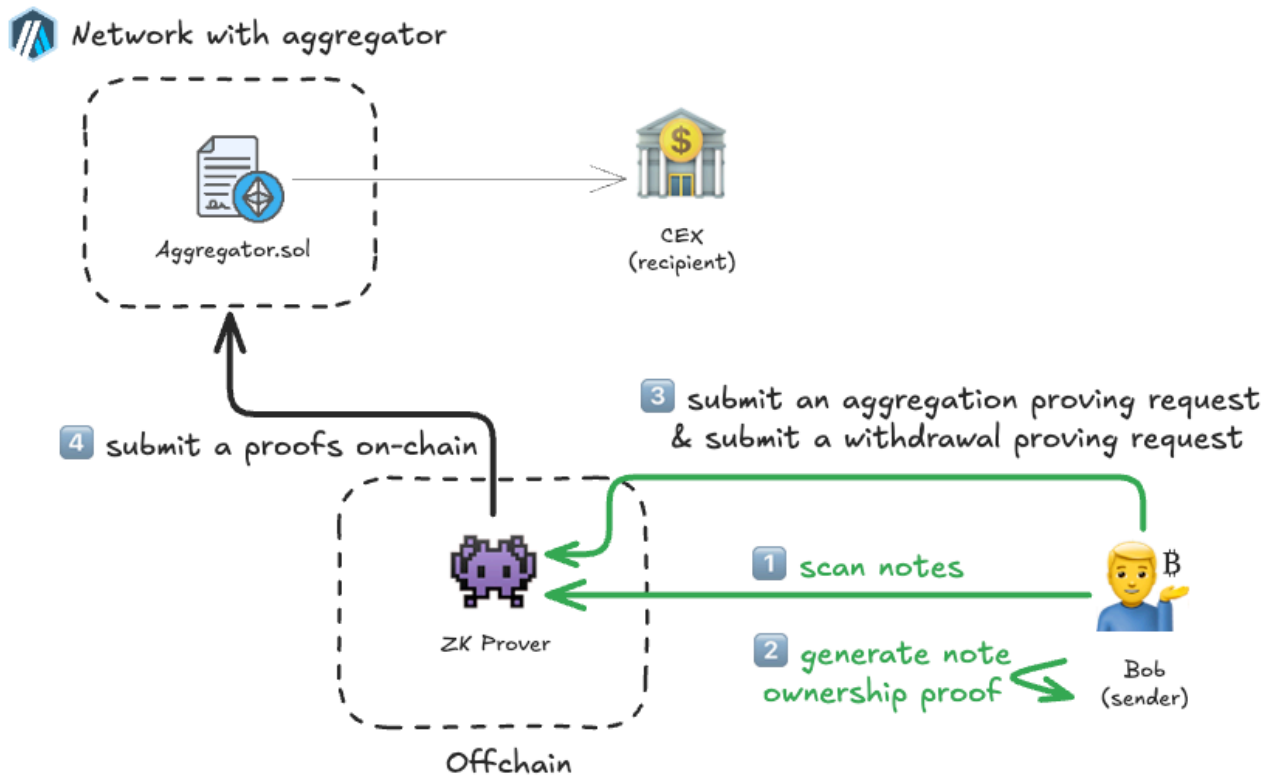
The simplified receiving funds privately flow is given in the following diagram:



The simplified sending funds privately flow is given in the following diagram:



The simplified unshielding funds privately flow is given in the following diagram:



The following text will present a simplified description of the Curvy components under the audit scope:

Ethereum smart contracts overview

- **Portal Factory** is a smart contract deployed on each network that maintains code integrity of every deployed Portal and has helper view functions to calculate the Portal address given the `ownerHash` and `recoveryAddress`.
- **Portal** is a smart contract deployed to the predetermined address to which the funds were previously sent. During deployment, funds are moved to Aggregator.
- **CurvyAggregator** is a smart contract that shields/unshields the funds and emits Notes about shielded funds.
- **CurvyVault** manages token operations and keeps the funds.

Workflow for receiving funds privately

- The sender enters the receiver's ENS into the wallet of choice.
- Curvy **ENS resolver** fetches the receiver's public key from the **Identity Register** (off-chain).
- Based on the nonce and the receiver's public key, the address to which the **Portal** smart contract will be deployed is determined.
- This tuple of information is called an announcement, and it is stored in the **Announcement Registry** (off-chain).
- **Curvy Portal Broadcaster** (off-chain) continuously syncs announcements from the **Announcement Registry**.
- After observing a **Portal** address that has received any of the supported funds, it deploys the **Portal** contract through the **Portal Factory**.
- During the deployment, funds are automatically bridged if needed and shielded inside the **Privacy Aggregator**.
- The **Curvy ZK Prover** (off-chain) continuously keeps in sync with the Notes emitted by **Aggregator**, which are indexed in the public **Notes Registry** (off-chain).
- After a shielding event is observed on **Aggregator**, the **ZK Prover** constructs the proof for the shielding of the new note. After successful proof verification on-chain, the new note is emitted, and a

new version of the resulting Notes tree is verified using on-chain ZK verifiers that are a part of the **Aggregator**.

- The receiver logs into his **Curvy App** (off-chain) and starts syncing the notes from the public **Notes Registry**. The **Curvy SDK** simultaneously scans the notes it has synced with the receiver's private keys to detect which notes are secretly in his ownership and to decrypt their balances.

Workflow for sending funds privately

- The sender queries the **Identity Register** for the receiver's public key.
- Using the **Curvy SDK**, a new note is created for which the receiver is the owner.
- Aggregation is requested for which sender notes are spent, and a new note for the receiver is created.
- **ZK Prover** generates a new aggregation proof and submits it on-chain.
- The receiver logs into his **Curvy App** and starts syncing the notes from the public **Notes Registry**. The **Curvy SDK** simultaneously scans the notes it has synced with the receiver's private keys to detect which notes are secretly in his ownership and to decrypt their balances.

Workflow for unshielding funds privately

- Owner syncs and scans notes from the public **Notes Registry**.
- After matching notes for ownership during scanning, the Curvy SDK generates local ownership proofs used to decrypt the note balances and the exact token.
- The owner then submits aggregation requests that merge multiple notes, followed by a subsequent unshielding request.
- After generating the proofs, the ZK Prover submits them on-chain to Aggregator.sol.
- Aggregator.sol, after verifying the withdrawal proof, automatically initiates the unshielding process from itself to the requested address.

Curvy 2026 Q1 Invariant Analysis

Smart Contract Deployment and Upgrade

Invariant 1. The deployment process could not lead to misconfiguration

Conclusion: This invariant holds. The process of deployment is managed by Hardhat Ignition, a declarative system for the deployment of smart contracts. During the deployment process, all the required configurations are set in the smart contracts, usually by calling *updateConfig* functions.

Invariant 2. Upgrading smart contracts will not create any storage layout issues

Conclusion: This invariant holds. Two smart contracts are upgradable: `CurvyAggregatorAlphaV6` and `CurvyVaultV6`. `PortalFactory` and `Portal` are not expected to be upgradable, based on the system architecture. `CurvyAggregatorAlphaV6` and `CurvyVaultV6` rely on OpenZeppelin's *UUPSUpgradeable* abstract contract. In both cases, the proxy is set up correctly. In the process of upgrading from version V1 to V6, the storage layout has been preserved.

Invariant 3. EOA addresses cannot be set where a smart contract is expected

Conclusion: This invariant does not hold. None of the functions that assign contract-role addresses perform an `extcodesize` check to verify the target is a deployed contract. Specifically:

- *updateConfig()* in the aggregator accepts zero-address-guarded but otherwise unchecked addresses for `insertionVerifier`, `aggregationVerifier`, `withdrawVerifier`, `curvyVault`, and `portalFactory`. Setting any verifier to an EOA would cause `commitDepositBatch`, `commitAggregationBatch`, or `commitWithdrawalBatch` to revert silently on the verifier call, halting all protocol progress.
- *setCurvyAggregatorAddress()* in the vault accepts any address. Setting it to an EOA would cause `vault.deposit()` to become permanently callable by that EOA address directly.
- *registerToken()* accepts any non-zero address with no code check, as covered in the previous invariant.
- `Portal` constructor accepts any non-zero address for `_recovery` with no code check – though recovery is intended to be an EOA or multisig, so this is by design.

Finding: [Missing Smart Contract address check](#)

PortalFactory and Portal smart contracts

Invariant 4. Portal addresses are deterministic and cannot be predicted without the correct inputs

Conclusion: This invariant holds. `PortalFactory` deploys all portals via `CREATE2` with a fixed salt (`keccak256("curvy-portal-factory-salt")`). The deployed address is a pure function of (`ownerHash`, `exitAddress`, `exitChainId`, `recovery`) plus the factory address. *getEntryPortalAddress* and *getExitPortalAddress* reproduce the same hash off-chain, meaning no two calls with different parameters can produce the same address, and the address cannot be front-run with different bytecode.

Invariant 5. LiFi calldata is validated before a portal is deployed and funded

Conclusion: This invariant holds. In *deployEntryBridgePortal*, the receiver field in the LiFi `bridgeData` is verified to match the pre-computed portal address, and the `destinationChainId` must equal `AGGREGATOR_CHAIN_ID` (Arbitrum, 42161). In *deployExitBridgePortal*, the receiver is similarly verified against `exitAddress`. When `exitChainId == block.chainid`, the operation is a same-chain swap and the calldata is validated via *extractGenericSwapParameters*; the `destinationChainId` check is intentionally omitted because no bridging occurs. When `exitChainId != block.chainid`, the operation is a cross-chain bridge, and both receiver

and destinationChainId are validated via *extractBridgeData*. In both cases, the correct LiFi extraction method is applied for the operation type.

Invariant 6. A portal can only be used for its intended operation once

Conclusion: This invariant holds. Every Portal contract carries a *_used* boolean protected by the *onlyOnce* modifier. *shield()* and *bridge()* both apply this modifier, making each portal single-use. The one exception is intentional: if *shield()* fails because *curvyVault.getTokenAddress()* reverts (unregistered token), *_used* is reset to *false* to allow a future retry, and this is by design.

Invariant 7. Deploying a portal and executing its operation are atomic

Conclusion: This invariant partially holds. In *deployEntryBridgePortal* and *deployExitBridgePortal*, the CREATE2 deployment and the subsequent *bridge()* call are in the same transaction with no try/catch; any failure in *bridge()* reverts the deployment entirely. In *deployShieldPortal*, however, the call to *shield()* wraps *curvyVault.getTokenAddress()* in a try/catch that silently returns on failure. In this case, the portal is left deployed and registered in *_registeredPortals*, but the note has not been shielded, and any funds sent via *callvalue()* are stranded in the portal until *recover()* is called.

Invariant 8. Only shielding portals deployed by PortalFactory can interact with the protocol

Conclusion: This invariant holds. *_registeredPortals* tracks shielding portals that have been deployed and approved by the factory, and this mapping is checked by the aggregator before accepting a shield operation.

Invariant 9. PortalFactory configuration cannot be set to zero or EOA addresses once configured

Conclusion: This invariant partially holds. The *updateConfig* function only updates *_curvyVaultProxyAddress*, *_curvyAggregatorAlphaProxyAddress*, and *_lifiDiamondAddress* when the provided value is non-zero, but there is no check that the address is not EOA.

Finding: [Missing Smart Contract address check](#)

Invariant 10. PortalFactory cannot be used for shielding unless both the vault and aggregator addresses are configured

Conclusion: This invariant holds. *deployShieldPortal* reverts with *UnsupportedShielding* if either *_curvyVaultProxyAddress* or *_curvyAggregatorAlphaProxyAddress* is *address(0)* (related to Invariant 8 partially holds). This ensures *shield()* is never called with zero addresses, which would cause silent misbehaviour rather than a clean revert.

Invariant 11. PortalFactory cannot be used for bridging unless the LiFi diamond address is configured

Conclusion: This invariant holds. Both *deployEntryBridgePortal* and *deployExitBridgePortal* revert with *UnsupportedBridging* if *_lifiDiamondAddress* is *address(0)* (related to Invariant 8 partially holds). No portal is deployed, and no funds are moved.

Invariant 12. A portal can never be deployed to the zero address

Conclusion: This invariant holds. All three deploy functions check the result of the CREATE2 opcode and revert with *DeploymentFailed* if *portalAddress == address(0)*. CREATE2 returns zero only if the deployment itself fails (e.g., out of gas, address collision), so this guard ensures the factory never proceeds to call *shield()* or *bridge()* on a non-existent contract.

Invariant 13. The LiFi receiver in the entry bridge calldata must match the pre-computed portal address

Conclusion: This invariant holds. *deployEntryBridgePortal* extracts the receiver from *bridgeData* via *extractBridgeData* and reverts with *InvalidLiFiReceiver* if it does not equal *getEntryPortalAddress(note.ownerHash, recovery)*. This ensures funds sent through LiFi arrive at exactly the portal that was deployed, not an arbitrary address.

Invariant 14. The LiFi destination chain in the entry bridge calldata must be the aggregator chain

Conclusion: This invariant holds. *deployEntryBridgePortal* reverts with *InvalidLiFiDestinationChain* if *extractedData.destinationChainId* != AGGREGATOR_CHAIN_ID (42161, Arbitrum). Entry bridge portals can only forward funds to the aggregator chain; bridging to any other chain is rejected.

Invariant 15. The LiFi receiver in the exit bridge calldata must match the intended exit address

Conclusion: This invariant holds. *deployExitBridgePortal* reverts with *InvalidLiFiReceiver* if the receiver extracted from *bridgeData* does not match *exitAddress*, for both the same-chain swap path and the cross-chain bridge path. Funds cannot be redirected to a different recipient by manipulating the bridge calldata.

Invariant 16. The LiFi destination chain in the exit bridge calldata must match the declared exit chain

Conclusion: This invariant holds. In the cross-chain path of *deployExitBridgePortal*, the function reverts with *InvalidLiFiDestinationChain* if *extractedData.destinationChainId* != *exitChainId*. The declared *exitChainId* and the chain encoded in the LiFi calldata must agree; a caller cannot specify one chain in the parameters while the calldata routes funds to another.

Invariant 17. A portal constructor will revert if given an invalid combination of owner hash and exit bridge data

Conclusion: This invariant holds. The Portal constructor enforces that a portal is either an entry portal or an exit portal. Any other combination reverts with *InvalidOwnerHashOrExitBridgeData*. This prevents creating a hybrid portal that could ambiguously act as both.

Invariant 18. A portal constructor will revert if given a zero recovery address

Conclusion: This invariant holds. The constructor reverts with *InvalidRecoveryAddress* if the recovery address is the zero address. Since recovery is the only address authorised to call *recover()*, deploying with *address(0)* would permanently strand any funds that end up in the portal without being shielded or bridged.

Invariant 19. shield() will revert if the note's owner hash does not match the portal's owner hash

Conclusion: This invariant holds. *shield()* reverts with *InvalidOwnerHash* if *note.ownerHash* is not the same as *_ownerHash*. Because *_ownerHash* is "set" into the portal's bytecode at construction and determines the CREATE2 address, this check ensures that the note being shielded corresponds exactly to the portal that was deployed for it.

Invariant 20. The bridge() will revert if the portal does not hold sufficient balance to cover the bridging amount

Conclusion: This invariant holds. For ERC-20 tokens, *bridge()* reverts with *InsufficientBalanceForLiFiBridging* if *token.balanceOf(address(this))* < *amount*. For native ETH, the same check is applied against *address(this).balance*. A portal can never attempt to bridge more than it actually holds.

Invariant 21. A failed LiFi call on the bridge() reverts the entire deployment transaction

Conclusion: This invariant holds. *bridge()* reverts with *BridgeCallFailed* if the low-level call to the LiFi diamond returns false. Since *deployEntryBridgePortal* and *deployExitBridgePortal* call *bridge()* with no try/catch, this

revert propagates up and rolls back the entire transaction, including the CREATE2 deployment. The portal is never left on-chain in a partially-used state. Funds are not at risk because the transaction is atomic.

CurvyAggregatorAlphaV6.sol

Invariant 22. `autoShield()` can only be called for tokens registered in the vault

Conclusion: This invariant holds. `autoShield()` calls `curvyVault.getTokenAddress(note.token)` without a try/catch. If the token is not registered in the vault, that call reverts, and the entire `autoShield()` call reverts with it. This is consistent with the Portal's try/catch around the same call — Portal catches this revert, resets `_used = false`, and returns without shielding, leaving the portal available for a retry once the token is registered.

Invariant 23. `commitDepositBatch()` can only succeed if all non-zero note IDs in the batch are pending

Conclusion: This invariant holds. For each non-zero `noteld` in `publicInputs`, `commitDepositBatch()` checks `_pendingIdsQueue[noteld]` and reverts with `NoteNotScheduledForDeposit` if it is not set. A deposit batch proof cannot be committed for notes that were never deposited through `autoShield()`, preventing fabricated or replayed notes from being inserted into the tree.

Invariant 24. `commitDepositBatch()` can only succeed if the submitted old notes tree root matches the current on-chain root

Conclusion: This invariant holds. `commitDepositBatch()` reverts with `InvalidNotesRoot` if `publicInputs[numPublicInputs - 2]` does not equal `_notesTreeRoot`. This ensures that the ZK proof was generated against the current state of the notes tree and not a stale or fabricated root.

Invariant 25. `commitAggregationBatch()` and `commitWithdrawalBatch()` can only succeed with a valid ZK proof

Conclusion: This invariant holds. `commitAggregationBatch()` calls `aggregationVerifier.verifyProof()` and reverts with `InvalidProof` if it returns false. `commitWithdrawalBatch()` calls `withdrawVerifier.verifyProof()` and reverts with `InvalidProof` if it returns false. Both tree roots are only updated if the proof is valid, ensuring the aggregation transition is cryptographically sound.

Invariant 26. `commitWithdrawalBatch()` nullifier root update is applied before vault withdrawals are executed

Conclusion: This invariant holds. `_nullifiersTreeRoot` is updated to `publicInputs[0]` before the withdrawal loop begins. This ordering means that even if a vault `withdraw()` call were to re-enter `commitWithdrawalBatch()`, the on-chain nullifier root would already reflect the spent nullifiers, causing the re-entrant call to fail the root check.

CurvyVaultV6.sol

Invariant 27. A token cannot be deregistered if it was never registered

Conclusion: This invariant holds. `deregisterToken()` reverts with `TokenNotRegistered` if the token is not previously registered. This also prevents deregistering ETH by a separate path since ETH is registered at `ETH_ID = 1` and would only fail this check if already deregistered.

Invariant 28. Deregistering a token does not result in loss of user funds held by the aggregator

Conclusion: This invariant does not hold. `deregisterToken()` sets `_tokenIdToTokenAddress[tokenId] = address(0)` with no check on the aggregator's balance for that token ID. Any funds held in `_balances[_curvyAggregator][tokenId]` at the time of deregistration become permanently frozen — `vault.withdraw()` reverts with `TokenNotRegistered` blocking ZK-proven withdrawals, and re-registering the same token address assigns a new incremented ID via `_numberOfTokens++` leaving the balance under the old ID unreachable.

Finding: [Deregistering a token does not check the vault balance](#)

Invariant 29. Only smart contract addresses can be registered as tokens

Conclusion: This invariant does not hold. `registerToken()` accepts any non-zero address that has not been previously registered. There is no check that `tokenAddress` contains contract code — an EOA address or a pre-computed address for a contract not yet deployed would be accepted without revert. If such an address were registered and a user deposited against it, the subsequent `IERC20(tokenAddress).safeTransferFrom()` call in `vault.deposit()` would revert since there is no contract to call, effectively blocking deposits for that token.

Finding: [EOA as tokenAddress](#)

Invariant 30. Fee collection reverts if the token is not registered

Conclusion: This invariant holds. `collectFees()` reverts with `TokenNotRegistered` if a token is not registered. The owner cannot collect fees for a token that has been deregistered or never existed, preventing a call to `safeTransfer` or an ETH transfer against a zero address.

Invariant 31. `collectFees()` reverts if the ETH transfer to the owner fails

Conclusion: This invariant holds. When collecting ETH fees, `collectFees()` performs a low-level call and reverts with `ETHTransferFailed` if it returns false. Because the revert rolls back the entire call, the zeroing of the balance is also undone — the owner's balance is fully restored, and the funds remain in the vault. The owner can retry the collection once the transfer condition is resolved.

Invariant 32. There is only one way to collect accumulated fees from the vault

Conclusion: This invariant does not hold. Two paths exist for the owner to extract fee balances from `_balances[owner()][tokenId]`. The intended path is `collectFees()`, which transfers the full balance with no fee deduction and is restricted to `onlyOwner`. The unintended path is `withdraw()`, which explicitly allows `msg.sender == owner()` and equally debits `_balances[owner()][tokenId]` at line 191 — however, it applies `withdrawalFee` on top, meaning the owner would pay a fee to collect their own fees, and only the aggregator address is credited with that secondary fee.

Finding: [Duplicated functionality for collecting fees](#)

Invariant 33. `deposit()` can only be called by the aggregator

Conclusion: This invariant holds. `deposit()` reverts with `NotCurvyAggregator` if the message sender is not `_curvyAggregator`. No external address, including the owner, can directly deposit into the vault. All deposits must flow through the aggregator, which enforces the ZK note queue.

Invariant 34. `deposit()` cannot credit a zero address

Conclusion: This invariant holds. `deposit()` reverts with `InvalidRecipient` if `to` is set to the zero address. Balances credited to `address(0)` would be permanently unrecoverable, so this guard prevents accidental loss of funds.

Invariant 35. deposit() of an ERC-20 token reverts if ETH is sent alongside it

Conclusion: This invariant holds. When *tokenAddress* is not *ETH_ADDRESS*, *deposit()* reverts with *ERC20TransferFailed* if any ETH is sent with the message. This prevents ETH from being silently stranded in the vault during an ERC-20 deposit, as the ETH would be credited to no one.

Invariant 36. deposit() of an ERC-20 token reverts if the token is not registered

Conclusion: This invariant holds. *deposit()* looks up if a token is registered and reverts with *TokenNotRegistered* if it is not. Tokens not registered by the owner cannot be deposited, ensuring only approved tokens enter the vault.

Invariant 37. deposit() of ETH reverts if the declared amount does not match msg.value

Conclusion: This invariant holds. When *tokenAddress* is *ETH_ADDRESS*, *deposit()* reverts with *ETHTransferFailed* if amount is not equal to the ETH sent with the message. The on-chain balance credited always equals the ETH actually received; there is no way to credit more or less than what was sent.

Invariant 38. The Deposit event accurately reflects the amount credited to the recipient's vault balance

Conclusion: This invariant does not hold. The Deposit event is emitted at *CurvyVaultV6.sol* with the original amount parameter — the gross pre-fee value. However, when *depositFee* != 0, the fee is deducted from *_balances[to][tokenId]*, meaning the actual net balance credited to to is *amount - feeAmount*, not *amount*. An off-chain observer or indexer consuming this event would overestimate the recipient's credited balance by exactly *feeAmount*. The invariant would hold if the event emitted the post-fee amount, or included both the gross amount and the fee amount as separate fields, allowing consumers to derive the true credited balance.

Finding: [Wrong data in the event](#)

Invariant 39. withdraw() can only be called by the aggregator

Conclusion: This invariant holds. *withdraw()* reverts with *NotCurvyAggregatorOrOwner* if the message sender is neither the *_curvyAggregator* nor the owner. The owner path exists for emergency forced withdrawals; all normal withdrawals initiated by users flow through the aggregator after ZK proof verification.

TODO FINDING

Invariant 40. withdraw() cannot send funds to a zero address

Conclusion: This invariant holds. *withdraw()* reverts with *InvalidRecipient* if *to* is set to the zero address. This mirrors the same guard in *deposit()* and prevents funds from being permanently destroyed.

Invariant 41. withdraw() reverts if the token ID is not registered

Conclusion: This invariant holds. *withdraw()* looks up if a token is registered and reverts with *TokenNotRegistered* if it is not. A deregistered or unknown token ID cannot be used to trigger a transfer.

Invariant 42. withdraw() reverts if the caller's vault balance is insufficient

Conclusion: This invariant holds. The subtraction *_balances[msg.sender][tokenId] -= amount* at uses Solidity 0.8 checked arithmetic and will revert with an underflow panic if amount exceeds the caller's balance. No explicit error is emitted, but the underflow revert is sufficient to prevent withdrawing more than is held.

Invariant 43. withdraw() reverts if the ETH transfer to the recipient fails

Conclusion: This invariant holds. When withdrawing ETH, *withdraw()* performs a low-level call and reverts with *ETHTransferFailed* if it returns false. The revert rolls back the entire call, including the balance deduction – the caller's balance is fully restored, and funds remain in the vault.

Invariant 44. Calling *recover()* never results in loss of funds

Conclusion: This invariant holds with one caveat. *recover()* always transfers the portal's full balance of the specified token – it reads *address(this).balance* or *token.balanceOf(address(this))* at call time and transfers the entire amount. For ETH, a failed transfer reverts via *require*, restoring state. For ERC-20, *safeTransfer* reverts on failure. In both cases, there is no partial transfer and no silent failure path – either the full balance reaches to or the entire call reverts. The one caveat is that to is caller-supplied with no zero address check: if the recovery address passes to *= address(0)*, *safeTransfer* to *address(0)* may succeed on some ERC-20 implementations that do not guard against it, permanently burning the tokens. For ETH, sending to *address(0)* would succeed at the EVM level. The invariant would hold unconditionally if a *require(to != address(0))* guard were added.

Finding: [Burning balance during recovery](#)

Invariant 45. The deposit fee is paid by the depositor; the withdrawal fee is paid by the recipient

Conclusion: This invariant holds. For deposits, the fee is deducted from the note value before the *noteId* is computed. The depositor sends *note.amount* but their note in the ZK system is only worth *note.amount - feeAmount* – the cost falls entirely on the party initiating the deposit. For withdrawals, the ZK proof encodes the full withdrawal amount as a public input, but *vault.withdraw()* only transfers *amount - feeAmount* to the recipient – the cost falls entirely on the party receiving the funds.

Invariant 46. *getTokenAddress()* reverts for unregistered or deregistered token IDs

Conclusion: This invariant holds. *getTokenAddress()* reverts with *TokenNotRegistered*

Invariant 47. *getTokenId()* reverts for unregistered or deregistered token addresses

Conclusion: This invariant holds. *getTokenId()* reverts with *TokenNotRegistered* if a token is not registered. Since ETH is pre-registered with *ETH_ID = 1* at initialisation, *getTokenId(ETH_ADDRESS)* will always succeed unless ETH is explicitly deregistered by the owner.

Bridging/Shielding flow

Invariant 48. A note can only enter the pending queue if the corresponding funds have been received by the vault

Conclusion: This invariant holds. *autoShield()* calls *vault.deposit()* at before computing and queuing the *noteId*. *vault.deposit()* either pulls the ERC-20 via *safeTransferFrom* or checks *msg.value == amount* for ETH – both revert if the funds are not actually transferred. Only after a successful deposit is the *noteId* added to *_pendingIdsQueue*. There is no path where a *noteId* enters the queue without the vault holding the corresponding funds.

Invariant 49. A note can only be inserted into the notes tree if it was previously queued via a successful deposit

Conclusion: This invariant holds. *commitDepositBatch* checks *_pendingIdsQueue[noteId]* for every non-zero note ID and reverts with *NoteNotScheduledForDeposit* if it is not set. The queue is only populated by *autoShield()*, which requires a registered portal caller and a successful vault deposit. The ZK proof is then verified against the insertion verifier before the notes tree root is updated. A note cannot appear in the tree without passing both the queue check and proof verification.

Invariant 50. Funds withdrawn from the vault always correspond to notes that have been nullified in the nullifiers tree

Conclusion: This invariant holds. `commitWithdrawalBatch` verifies the ZK withdrawal proof against `withdrawVerifier` before updating `_nullifiersTreeRoot` and executing any vault withdrawals. A withdrawal cannot be processed without a valid proof that the corresponding nullifiers have been consumed, preventing double-spending across multiple `commitWithdrawalBatch` calls.

Invariant 51. A note shielded through `deployShieldPortal` can only be withdrawn to the address encoded in the withdrawal proof

Conclusion: This invariant holds. The withdrawal recipient is taken directly from `publicInputs[3 + maxWithdrawals + i]` and passed to `vault.withdraw()`. This value is part of the public inputs bound to the ZK proof verified — if the proof is valid, the recipient is exactly what was committed to in the ZK circuit. No contract-level access control is needed because the ZK proof itself enforces who receives the funds.

Invariant 52. A failed shield attempt leaves the portal recoverable, but does not credit any note to the queue

Conclusion: This invariant holds. If `curvyVault.getTokenAddress()` reverts in `Portal.shield()`, the function catches the error, emits `ShieldingFailed`, resets `_used = false`, and returns without calling `autoShield()`. Since `autoShield()` is never reached, no `noteId` is added to `_pendingIdsQueue` in the aggregator, and no funds are transferred to the vault. Any funds sent via `callvalue()` during deployment remain in the portal and are recoverable via `Portal.recover()` by the designated recovery address.

Invariant 53. Fees accumulated across deposits and withdrawals can only be collected by the vault owner, and only for registered tokens

Conclusion: This invariant holds. Fees are credited to `_balances[owner()][tokenId]` during `deposit()` and `withdraw()`. The only function that debits the owner's balance and transfers funds out is `collectFees()`, which is only `owner` and reverts with `TokenNotRegistered` if the token ID is invalid. No other address can access the fee balance, and fees for deregistered tokens remain in the vault until the owner re-registers the token.

Invariant 54. The owner cannot unilaterally steal user funds without access to the ZK layer, but holds sufficient privilege to do so through contract manipulation

Conclusion: This invariant does not hold. The owner cannot steal funds through a single direct call — `vault.withdraw()` only debits `_balances[msg.sender][tokenId]`, which for the owner is limited to accumulated fees. However, multiple indirect vectors exist:

1. Fee manipulation: `setFeeAmount()` imposes no upper bound on `depositFee` or `withdrawalFee` (`CurvyVaultV6.sol:113–118`). Setting either to 10000 (100%) would redirect the entirety of every deposit or withdrawal to `_balances[owner()]`, draining user funds through normal protocol operation.
 2. Malicious verifier injection: `updateConfig()` allows the owner to replace `insertionVerifier`, `aggregationVerifier`, or `withdrawVerifier` with a contract that accepts any proof (`CurvyAggregatorAlphaV6.sol:85–98`). Combined with `reset()` to set arbitrary tree roots, the owner could construct fraudulent withdrawal proofs and drain the vault.
 3. Malicious aggregator substitution: `setCurvyAggregatorAddress()` allows the owner to point the vault to any address (`CurvyVaultV6.sol:120–123`). A malicious aggregator contract could call `vault.withdraw()` freely, as the only guard is `msg.sender == _curvyAggregator`.
 4. UUPS upgrade: Both `CurvyAggregatorAlphaV6` and `CurvyVaultV6` are UUPS upgradeable with `_authorizeUpgrade` gated solely by `onlyOwner`. The owner can upgrade either contract to an arbitrary implementation with no timelocks or multisig requirements.
- The system therefore, requires full trust in the owner. Users' funds are safe only under the assumption that the owner key is not compromised and the owner does not act maliciously.

Findings: [No upper limit for the fee, EOA takeover](#)

Curvy 2026 Q1 Findings

Title	Type	Severity	Status
No upper limit for the fee	IMPLEMENTATION	MEDIUM	RESOLVED
Deregistering a token does not check the vault balance	IMPLEMENTATION	MEDIUM	RESOLVED
EOA takeover	IMPLEMENTATION	MEDIUM	RESOLVED
Burning balance during recovery	IMPLEMENTATION	MEDIUM	RESOLVED
Reset function	IMPLEMENTATION	MEDIUM	RESOLVED
Missing Smart Contract address check	IMPLEMENTATION	MEDIUM	RESOLVED
Lost gas for transaction payment	IMPLEMENTATION	LOW	RESOLVED
Short-circuit guard bug/operator precedence bug	IMPLEMENTATION	LOW	RESOLVED
No Error Handling for Invalid JSON	IMPLEMENTATION	LOW	RESOLVED
Missing Null/Undefined Return Handling	IMPLEMENTATION	LOW	RESOLVED
No Validation of Address Format	IMPLEMENTATION	LOW	RESOLVED
Duplicated functionality for collecting fees	IMPLEMENTATION	LOW	RESOLVED
Wrong data in the event	IMPLEMENTATION	LOW	RESOLVED
EOA as tokenAddress	IMPLEMENTATION	LOW	RESOLVED
Collecting zero fees	IMPLEMENTATION	LOW	RESOLVED
Gas optimization	OPTIMIZATION ISSUE	LOW	RESOLVED
Moving constant to JSON	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED
Deprecated fields	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED
LiFi error message not propagated	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED
No way to query which portals were deployed and when	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED
Inconsistent Parameter Naming	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED
Modifier instead of error	CODE IMPROVEMENT	INFORMATIONAL	RESOLVED

Curvy 2026 Q1 No upper limit for the fee

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

FEE_DENOMINATOR is 10000. Fee is applied as:

`uint256 feeAmount = (amount * depositFee) / FEE_DENOMINATOR;`

Scenario:

Setting the fee to 10000 would equal to 100% fee, and the entire amount would be transferred to the contract owner.

Recommendation:

```
uint96 private constant MAX_FEE = 1000; // 10% maximum
function setFeeAmount(CurvyTypes.FeeUpdate calldata feeUpdate) external onlyOwner {
    if (feeUpdate.depositFee > MAX_FEE) revert FeeTooHigh();
    if (feeUpdate.withdrawalFee > MAX_FEE) revert FeeTooHigh();
    depositFee = feeUpdate.depositFee;
    withdrawalFee = feeUpdate.withdrawalFee;
    emit FeeChange(feeUpdate);
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/8b51f7f97e91a2d4ae866b6adb96f8aac5af0c45>

Curvy 2026 Q1 Deregistering token does not check the vault balance

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

The `deregisterToken` function could deregister a token that the Vault still holds. If the token is deregistered, there is no implementation that would enable the recovery of those tokens.

Recommendation:

```
function deregisterToken(address tokenAddress) external onlyOwner {
    uint256 tokenId = _tokenAddressToTokenId[tokenAddress];
    if (tokenId == 0) revert TokenNotRegistered();
    // Prevent deregistering a token that still has funds in the vault
    uint256 vaultBalance = IERC20(tokenAddress).balanceOf(address(this));
    if (vaultBalance != 0) revert TokenHasOutstandingBalance();
    _tokenAddressToTokenId[tokenAddress] = 0;
    _tokenIdToTokenAddress[tokenId] = address(0);
    emit TokenDeregistered(tokenAddress, tokenId);
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/22deea230a1e6c12364f9b15cbffaff230bdb83c>

Curvy 2026 Q1 EOA takeover

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- all upgradable smart contracts
- all onlyOwner functions

Description:

The owner is set to EOA, this gives a "sole" person the possibility of upgrading smart contracts and using that to commit fraud.

Scenario:

A malicious implementation could:

- Add a function sweepAll(address to) that calls IERC20.transfer(to, balance) directly on the vault's held ERC20 tokens
- Re-route the withdraw() function to send all funds to the owner
- Drain ETH via a selfdestruct or direct transfer

Recommendation:

Switch owner from EOA to multisig.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/72b55084d33d5011b468db8ce4b784dba4a473a3>

Curvy 2026 Q1 Burning balance during recovery

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/portal/Portal.sol](#)

Description:

The `recover` function is used to recover the balance. One of the parameters is the address to which balance should be recovered. There is no check that the address is a non-zero address.

Scenario:

Recovering to zero address would burn the balance.

Recommendation:

```
if (to == address(0)) revert InvalidRecoveryAddress()
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/c43f9eb70c20f690e598177189dd6bf68a31fcc0>

Curvy 2026 Q1 Reset function

RESOLVED Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/aggregator-alpha/CurvyAggregatorAlphaV6.sol](#)

Description:

Existence of reset function.

Recommendation:

To be removed in production.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/f46feed442d6412cf84c427fd4b79a8b512c6df7>

Curvy 2026 Q1 Missing Smart Contract address check

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	MEDIUM
Impact	HIGH
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/aggregator-alpha/CurvyAggregatorAlphaV6.sol](#)
- [contracts/portal/PortalFactory.sol](#)

Description:

Throughout the smart contracts, whenever an address is being passed as a parameter, it is correctly checked that it is not a zero address. However, in cases when it is expected that the parameter is an address of the smart contract, an additional check that it is indeed a smart contract and not EOA is not being made.

Problem Scenarios:

In the case of `CurvyAggregatorAlphaV6` and `PortalFactory`, setting an EOA as a smart contract address in `updateConfig` for either `insertionVerifier`, `aggregationVerifier`, `withdrawVerifier`, `curvyVault`, or `portalFactory` in `CurvyAggregatorAlphaV6` and `_curvyVaultProxyAddress`, `_curvyAggregatorAlphaProxyAddress`, or `_lifiDiamondAddress` in `PortalFactory` would lead to protocol malfunction.

The likelihood of this happening is low due to the `onlyOwner` modifier on the `updateConfig` function.

Recommendation:

To mitigate this risk, wherever a smart contract address is being passed as an argument, a check that it is not a zero address should be replaced with a check that it is a smart contract address.

```
someaddress.code.length > 0;
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/db4e912e9a44766c594500845a0fbe43a7494b0c>

Curvy 2026 Q1 Lost gas for transaction payment

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/portal/Portal.sol](#)

Description:

In the `recover` function, there is no check if tokenAddress is the zero address, which could represent native tokens (as there is in the `bridge` function).

Scenario:

If tokenAddress is set to the zero address when calling the `recover` function, it will result in the zero address being recognized as `IERC20`. There would be no funds lost, but the gas for calling this transaction will be spent.

Recommendation:

Extend the check for native token to include the zero address, as already implemented in the `bridge` function.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/39e337ac4532c60300cd04712ab571f2009435e7>

Curvy 2026 Q1 Short-circuit guard bug/operator precedence bug

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [ignition/modules/utls/parameters.ts](#)

Description:

Line:60

```
if (parameters[environment] === undefined || !parameters[environment][parameterName]
=== undefined) {
    throw new Error(`Parameter ${parameterName} not found for environment $
{environment}`);
}
```

Probably as a result of a short-circuit, there is a check that is never true

!parameters[environment][parameterName] === undefined

! takes precedent, and if *parameters[environment][parameterName]* is *undefined*, it will be converted to *false*, and then negated to *true*, and *true* is never equal to *undefined*. Similarly, *false* is never equal to *undefined*.

Problem Scenarios:

In PortalFactory.ts and in deploy.ts, `getEnvironmentParameter` is called, which might return the owner, and in case it is not set, it is not checked in those deployment scripts.

Recommendation:

Remove *!*.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/516db43c5022c5caaffee546f59c2fa427a36105>

Curvy 2026 Q1 No Error Handling for Invalid JSON

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [ignition/modules/utis/parameters.ts](#)

Description:

In `readParameters` function, there is no check for Malformed JSON.

```
const rawData = fs.readFileSync(filePath, "utf-8");
return JSON.parse(rawData);
```

Problem Scenarios:

If either `network-parameters.json` or `environment-parameters.json` contains invalid JSON, the deployment will crash with a cryptic error:

SyntaxError: Expected property name or '}' in JSON at position 1 (line 1 column 2)

This error message is not helpful for deployment debugging.

Recommendation:

Surround by a try/catch block.

```
try {
  return JSON.parse(rawData);
} catch (error) {
  throw new Error(
    `Failed to parse ${filename}: ${error instanceof Error ? error.message :
    String(error)}`
  );
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/2f2f58cba142d11fe4170ce77e375f421cf6866f>

Curvy 2026 Q1 Missing Null/Undefined Return Handling

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [ignition/modules/utis/parameters.ts](#)

Description:

Line 50, Line 64

If a parameter is explicitly set to null or undefined in the JSON, it will be returned as-is, not caught by the validation checks:

```
{
  "test": {
    "owner": null
  }
}
```

Problem Scenarios:

In case the parameter is set in the environment, but the value is null or undefined, this will be accepted by the script and result in an error later on instead of being caught right away.

Recommendation:

```
const value = parameters[environment][parameterName];
if (value === null || value === undefined) {
  throw new Error(
    `Parameter '${parameterName}' is null or undefined for environment '${environment}'`
  );
}
return value;
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/6dfda3b7a32f1fe7d618157dd1bffb83e5a02ee6>

Curvy 2026 Q1 No Validation of Address Format

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [ignition/modules/utils/parameters.ts](#)
- [ignition/modules/Deployment.ts](#)
- [ignition/modules/MainDeployment.ts](#)

Description:

After reading the owner and LiFiDiamondAddress from parameters, there is no check if those values are valid addresses.

Problem Scenarios:

Any string could be passed as an address and only be caught later in the process of deployment.

Recommendation:

```
if (!lifiDiamondAddress?.startsWith('0x') || lifiDiamondAddress.length !== 42) {
  throw new Error("Invalid lifiDiamondAddress format");
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/614abf4302020b1d6e973a2480f716c409316f60>

Curvy 2026 Q1 Duplicated functionality for collecting fees

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

The owner can collect fees by calling the `withdraw` or `collectFees` functions.

Recommendation:

Set onlyCurvyAgregator to the `withdraw` function.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/868ef483feecf8be65ac3edfbc49e1cc921914e>

<https://github.com/0xCurvy/contracts/commit/5187c8d585a362273f3e40459506d75876a63794>

<https://github.com/0xCurvy/contracts/commit/d232edba1f4515f29a0fd499d91f1238113e00bc>

Curvy 2026 Q1 Wrong data in the event

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

In the `function deposit()` `Deposit` event is emitted. In this event amount is emitted that could be the amount before the fee is deducted.

Scenario:

Recommendation:

Emit the truly deposited amount.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/19f8d4045bdc8a8805635cee2b494f886580a046>

Curvy 2026 Q1 EOA as tokenAddress

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

When a token is registered, there is no check that the address is, in fact, a smart contract address

Scenario:

The owner could make a mistake and register an address that is not an ERC20 address. This could be resolved by checking if the address is an address of a smart contract; furthermore, additional check could be performed by calling some function of ERC20, like `totalSupply()`.

Recommendation:

```
if (tokenAddress.code.length == 0) revert NotAContract();
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/10dd9b234cc16053076a555c8ac21a401e0a93a8>

Curvy 2026 Q1 Collecting zero fees

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATION
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

When collecting fees, there is no check that there is in fact fees to collect

Scenario:

The owner might call the collectFees function at the point of time when there are no fees to collect, resulting in zero transfer and loss of a bit of gas.

Recommendation:

amount > 0

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/d80a5c2d2bb2525cec0c3a2b0fd94a1625969e48>

Curvy 2026 Q1 Gas optimization

Project	Curvy 2026: Q1: Curvy Protocol
Type	IMPLEMENTATIONAL
Severity	LOW
Impact	LOW
Likelihood	LOW
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

In the deposit function, there are two calls to storage value, and the same effect could be achieved with one.

```
// Mint wrapped tokens
_balances[to][tokenId] += amount;

// Collect fees if they are set
if (depositFee != 0) {
    uint256 feeAmount = (amount * depositFee) / FEE_DENOMINATOR;
    _balances[to][tokenId] -= feeAmount;
    _balances[owner()][tokenId] += feeAmount;
}
```

Scenario:

Recommendation:

```
// Collect fees if they are set
if (depositFee != 0) {
    uint256 feeAmount = (amount * depositFee) / FEE_DENOMINATOR;
    _balances[to][tokenId] += amount - feeAmount;
    _balances[owner()][tokenId] += feeAmount;
} else {
    _balances[to][tokenId] += amount;
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/aa180b3661b743beeb4945d302a7dbd09499300d>

Curvy 2026 Q1 Moving constant to JSON

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [ignition/modules/PortalFactory.ts](#)

Description:

CREATEX_ADDRESS address is hardcoded in PortalFactory.rs

Recommendation:

Move hardcoded CreateX address to network-parameters.json

Add error messages if the artifact is malformed

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/2c394a9ec5e288760a8ba18d260e5ff958634de7>

Curvy 2026 Q1 Deprecated fields

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV5.sol](#)
- [contracts/vault/CurvyVaultV6.sol](#)

Description:

In these two versions of contracts, there are two deprecated fields:

```
// DEPRECATED: This is no longer used, but because of storage layout it *MUST* not be
deleted
uint96 public transferFee;
```

```
// DEPRECATED: This is no longer used, but because of storage layout it *MUST* not be
deleted
bytes32 private constant CURVY_META_TRANSACTION_TYPE_HASH = keccak256(
    "CurvyMetaTransaction(uint256 nonce,address from,address to,uint256
tokenId,uint256 amount,uint256 gasFee,uint8 metaTransactionType)"
);
```

To avoid confusion transferFee field could be renamed to __deprecated with a comment containing the old field name.

CURVY_META_TRANSACTION_TYPE_HASH is a constant and can safely be removed with no issue regarding storage layout.

Recommendation:

Rename transferFee.

Remove CURVY_META_TRANSACTION_TYPE_HASH.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/d47eea46ebdea41aca762f01b52ca1d7b566603d>

Curvy 2026 Q1 LiFi error message not propagated

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/portal/Portal.sol](#)

Description:

In the `bridge` function, when calling `lifiDiamondAddress.call(bridgeData)`, only success is picked up as a result and correctly acted on, but no error in case it is provided by the call.

Scenario:

In case of a failed call to `lifiDiamondAddress.call(bridgeData)`, the root error is not propagated; instead, a revert happens with `BridgeCallFailed`

Recommendation:

Propagated the error if it exists.

```
bool success, bytes memory result) = lifiDiamondAddress.call(bridgeData);
if (!success) {
    if (result.length > 0) {
        // Propagate actual revert from LiFi
        assembly {
            revert(add(32, result), mload(result))
        }
    }
    revert BridgeCallFailed();
}
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/7b4352555acb078a1b2ca041ae97eea5e6f3e7c0>

Curvy 2026 Q1 No way to query which portals were deployed and when

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/portal/PortalFactory.sol](#)

Description:

When deploying Shield or Bridge portals, no events are emitted.

Scenario:

After deployment of a portal, there is:

- No Audit Trail
- No way to query which portals were deployed and when
- ...

Recommendation:

Emit events after deployment.

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/8e8aac1107675be2ad66b2e72b081cf466c045bb>

Curvy 2026 Q1 Inconsistent Parameter Naming

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/portal/Portal.sol](#)

Description:

Typo

Scenario:

```
function shield(
CurvyTypes.Note memory note,
address curvyAgrgegatorAlphaProxyAddress,
address curvyVaultProxyAddress
) external;
```

Recommendation:

curvyAggregatorAlphaProxyAddress

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/3ce3e67bab433af9f9d3f7e6289ca5b47bd7644c>

Curvy 2026 Q1 Modifier instead of error

Project	Curvy 2026: Q1: Curvy Protocol
Type	CODE IMPROVEMENT
Severity	INFORMATIONAL
Impact	NONE
Likelihood	NONE
Status	RESOLVED
Issue(s)	

Involved Artifacts:

- [contracts/vault/CurvyVaultV6.sol](#)

Description:

Calling `withdraw` function has a check that the caller is `CurvyAgregator` or `Owner`. This is not clear from the function signature. Instead of this check in the function, a modifier could be used

Scenario:

Recommendation:

```
OnlyCurvyAgregatorOrOwner OnlyCurvyAgregatorOrOwner
```

Status:

This issue has been resolved through:

<https://github.com/0xCurvy/contracts/commit/10be7221f38663368d143f111429d707f0018f04>

Curvy 2026 Q1 Findings Classification

Finding Categories

- *Protocol*: A flaw or problem in an abstract protocol or algorithm.
- *Implementation*: A problem with the source code. For example, a bug, a divergence from a specification, or a poor choice of data structure.
- *Code improvement*: A problem that impacts the extent to which the project is maintainable and understandable by developers in the long term.
- *Documentation*: A lack of documentation, or insufficient clarity, accuracy, understandability, or readability of existing documentation.

Severity Levels

- *Informational*: The issue does not pose an immediate risk (it is subjective in nature). Findings with Informational severity is typically suggestions around best practices or readability.
- *Low*: The issue is objective in nature, but the security risk is relatively small or does not represent a security vulnerability.
- *Medium*: The issue is a security vulnerability that may not be directly exploitable or may require certain complex conditions in order to be exploited.
- *High*: The issue is an exploitable security vulnerability.
- *Critical*: The issue that disrupts functionality, security, or performance that leads to system failure, data loss, or security breaches.

Finding Severity Categories

Finding severity is derived from findings, Impact, and Likelihood as follows:

SEVERITY		LIKELIHOOD			
		NONE	LOW	MEDIUM	HIGH
IMPACT	NONE	INFORMATIONAL	INFORMATIONAL	INFORMATIONAL	INFORMATIONAL
	LOW	INFORMATIONAL	LOW	LOW	MEDIUM
	MEDIUM	INFORMATIONAL	LOW	MEDIUM	HIGH
	HIGH	INFORMATIONAL	MEDIUM	HIGH	CRITICAL

Curvy 2026 Q1 Disclaimer

This report is subject to the terms and conditions (including, without limitation, description of services, confidentiality, disclaimer, and limitation of liability, etc.) set forth in the associated Services Agreement. This report, provided in connection with the Services set forth in the Services Agreement, shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement.

This audit report is provided on an “as is” basis, with no guarantee of the completeness, accuracy, timeliness, or results obtained by use of the information provided. Eternal has relied upon information and data provided by the client, and is not responsible for any errors or omissions in such information and data or results obtained from the use of that information or conclusions in this report. Eternal makes no warranty of any kind, express or implied, regarding the accuracy, adequacy, validity, reliability, availability, or completeness of this report. This report should not be considered or utilized as a complete assessment of the overall utility, security, or bug-free status of the code.

This audit report contains confidential information and is only intended for use by the client. Reuse or republication of the audit report other than as authorized by the client is prohibited. This report is not, nor should it be considered, an “endorsement”, “approval,” or “disapproval” of any particular project or team. This report is not, nor should it be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts with Eternal to perform a security assessment.

This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor does it provide any indication of the client’s business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should it be leveraged as investment advice of any sort.

Blockchain technology and cryptographic assets, in general, and by definition, present a high level of ongoing risk. Client is responsible for its own due diligence and continuing security in this regard.